BMAD DCA — AI Agent Coverage

Multi-agent code intelligence for the Adobe / Java middleware stack · 2026-07-10 · branch feature/aem-ams-acs

Tech stack	Audit	Generation	Impact	Test Coverage
AEMaaCS	DONE	DONE	DONE	DONE
AEM AMS	DONE	DONE	DONE	DONE
Adobe Commerce PaaS	DONE	DONE	DONE	DONE
Adobe Commerce SaaS	DONE	DONE	DONE	DONE
Adobe App Builder	DONE	DONE	DONE	DONE
Sling-12 / Shaft	DONE	DONE	DONE	DONE
Spring Boot	DONE	DONE	DONE	DONE
Adobe EDS	DONE	DONE	DONE	DONE
EDS + Commerce	DONE	DONE	DONE	DONE

36 of 36 stack x agent cells = DONE (all 9 company tech stacks, all 4 agents)

- Audit: every stack emits one identical standardized report + CHANGE-LOG (incl. App Builder + Commerce-SaaS webhook signature checks).
- Generation: deterministic scaffolders for every stack (generated PHP passes php -l, Java is javac-valid) + LLM/MCP packs.
- Impact: a Proofhub bug export or a BRD is traced to impacted code with reverse-dependency blast radius (Input Traceability sheet).
- Test Coverage: coverage-gap analysis across every stack.

Agents at a glance

Agent	What it does	Standard outputs (A/B/C)
Audit 9 of 9 stacks	Two-tier static scanners (tree-sitter AST) + LLM rule packs !' one standardized report + CHANGE-LOG.	A: yes B: yes C: opt-in (--create-branch)
Generation 9 of 9 stacks	Deterministic scaffolder (real files, php -l / javac valid) + LLM/MCP resource packs.	A: yes B: yes C: not wired
Impact Analysis 9 of 9 stacks	Proofhub bug export / BRD -> traced to impacted code with reverse-dependency blast radius.	A: yes B: yes C: not wired
Test Coverage 9 of 9 stacks	Per-stack coverage-gap analysis (source<->test matching, priority scoring) -> standardized report.	A: yes B: yes C: not wired

Full per-stack engine/rule detail is in the companion workbook DCA-Agent-Coverage.xlsx (one sheet per agent). Two documented depth caveats: Impact tracing is heuristic (identifier + reverse-reference, evidence listed per finding); test-coverage % is file/class matching, not JaCoCo/nyc line-branch.